King Salman International University

Faculty of Computer Science and Engineering

Project Title

Intelligent Robot for Flame Location Detection

A Thesis Submitted to Computer Engineering Department
Faculty of Computer Science & Engineering, KSIU

In Partial Fulfillment of **B.Sc** Degree in Computer Engineering
Computer Engineering Program

By

Bassel Ashraf Elbahnasy

Supervisors

Dr. Mahmoud Zaki

Assistant Professor, Faculty of Computer Science & Engineering

King Salman International University

EGYPT 2026

I. Abstract

This thesis details the architectural overhaul, low-level hardware abstraction, and decentralized software integration of **Phoenix**, an intelligent mobile firefighting robot engineered for autonomous hazard mitigation, real-time flame localization, and adaptive path planning within unknown, structurally compromised environments.

A major contribution of this work is the complete migration of the robot's navigation framework from a static, pre-saved map topology utilizing simulated "fake" odometry to an entirely reactive, encoder-free **Dynamic Mapping and Real-Time Localization Stack**. Because the physical 4-wheel drive skid-steer chassis lacks hardware wheel encoders, traditional wheel-odometry feedback was unavailable, causing severe open-loop positional drift during high-friction turning maneuvers. To resolve this baseline constraint without structural or hardware overhead, a planar laser odometry estimation engine (rf2o_laser_odometry) was integrated to process raw distance data over a high-speed hardware UART serial pipeline from an onboard Okdo Direct Time-of-Flight (DTOF) LiDAR. By computing precise scan-match differentials across consecutive spatial arrays, the node continuously publishes the odom -> base_link transform tree completely independent of physical wheel encoders or a pre-existing map baseline.

These live spatial displacements are fed directly into the slam_toolbox online asynchronous mapping engine to generate 5 cm high-resolution environmental occupancy grids on the fly, eliminating the need to save, pre-load, or configure static map boundaries.

To minimize mobile power consumption and prevent thermal throttling on the mobile edge node (Raspberry Pi 4 Model B), the platform utilizes a distributed split-node computing topography linked via a localized, low-latency Mosquitto MQTT broker running over WebSocket and standard TCP channels. High-intensity computer vision pipelines—including a custom PyTorch Global Fire Detection Network (GFD-Net) and dual Keras binary classification layers—are fully offloaded to an external stationary laptop station. Real-world 3D spatial coordinate projection is achieved by tracking a rigid 4x4 ArUco fiducial chassis array with OpenCV, solving the Perspective-n-Point (PnP) problem to build real-time homogeneous extrinsic transformation matrices ($T_{\text{camera}}^{\text{robot}}$).

Target coordinate JSON payloads are streamed asynchronously to the mobile robot to command the ROS 2 Navigation Stack (Nav2). Upon successful arrival at the target safe-stopping boundary, an onboard control node activates an opto-isolated relay to energize a

high-pressure 24V suppression pump loop and direct an adjustable pan-tilt nozzle assembly. The software control nodes, velocity smoothing interpolation profiles, and sensor-fusion stacks were rigorously validated through high-fidelity Gazebo Harmonic simulations prior to physical hardware implementation, confirming excellent trajectory generation and obstacle avoidance in high-risk industrial layouts.

II. Acknowledgments

First and foremost, I would like to express my sincere gratitude and deepest respect to my supervisor, **Dr. Mahmoud Zaki**, Assistant Professor at the Faculty of Computer Science & Engineering, King Salman International University (KSIU), for his invaluable guidance, patience, and continuous academic support throughout the development of this graduation project. His technical insights, constructive critiques, and encouragement were instrumental in helping me navigate the complexities of this research and successfully complete the platform's architectural migration.

I would also like to extend my appreciation to **Dr. Ayman Elshabrawy**, Program Director, and the entire faculty of the Computer Engineering Department at KSIU for providing the academic foundation, resources, and environment necessary to turn this project into a reality. Finally, I am deeply grateful to my family and peers for their unwavering support and encouragement during my undergraduate studies.

III. Table of Contents

IV. Table of Figures

V. Table of Abbreviations

Abbreviation	Expanded Definition / Meaning
API	Application Programming Interface
B.Sc.	Bachelor of Science
bps	Bits Per Second
CNN	Convolutional Neural Network
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
DC	Direct Current
DMA	Direct Memory Access
DTOF	Direct Time-of-Flight
EEPROM	Electrically Erasable Programmable Read-Only Memory
EKF	Extended Kalman Filter
EMF	Electromotive Force (Back-EMF)
EMI	Electromagnetic Interference

Abbreviation	Expanded Definition / Meaning
EGP	Egyptian Pound
GFD-Net	Global Fire Detection Network
GND	Ground Reference Electrical Common
GPIO	General-Purpose Input/Output
HTTP	Hypertext Transfer Protocol
I2C	Inter-Integrated Circuit Communication Bus
IMU	Inertial Measurement Unit
IPPE	Infinitesimal Planar Pose Estimation Solver
JPEG	Joint Photographic Experts Group
JSON	JavaScript Object Notation
Keras	High-Level Neural Networks API (Running over TensorFlow)
KSIU	King Salman International University
LiDAR	Light Detection and Ranging

Abbreviation	Expanded Definition / Meaning
Li-Po	Lithium-Polymer Chemistry Battery
mAh	Milliampere-Hour Battery Capacity Metric
MEMS	Micro-Electromechanical Systems
MQTT	Message Queuing Telemetry Transport Protocol
Nav2	ROS 2 Navigation 2 Stack Architecture
PnP	Perspective-n-Point Spatial Matrix Solver
PWM	Pulse-Width Modulation Duty Cycle Control
PyTorch	Open-Source Deep Learning Tensor Framework
QoS	Quality of Service Message Management Parameters
RGB	Red, Green, Blue Color Space Channel Array
ROS 2	Robot Operating System 2 (Humble/Jazzy Ecosystems)
RPM	Rotations Per Minute Mechanical Threshold
RTSP	Real-Time Streaming Protocol

Abbreviation	Expanded Definition / Meaning
RX	Receiver Pin / Input Telemetry Line
SLAM	Simultaneous Localization and Mapping
SSH	Secure Shell Cryptographic Network Protocol
TensorFlow	End-to-End Open Source Machine Learning Platform
TF	Coordinate Transform Tree Framework
TX	Transmitter Pin / Output Telemetry Line
UART	Universal Asynchronous Receiver-Transmitter Serial Interface
URDF	Unified Robot Description Format Structural File
VCC	Voltage Common Collector / Positive Power Rail Input

1. Chapter 1: Introduction

1.1 Project Background and Industrial Motivation

Fire hazards inside industrial environments, high-capacity multi-level logistics warehouses, and petrochemical processing containment zones present catastrophic risks to human life, valuable physical inventory, and long-term structural integrity. Traditional fixed fire protection frameworks—such as localized ceiling smoke alarms and overhead sprinkler networks—frequently exhibit structural limitations, including substantial activation latency, vulnerability to visual blind spots, and untargeted deployment that inflicts severe collateral water damage to high-value assets across unignited zones. Furthermore, deploying human emergency containment teams into zones filled with toxic gases, volatile chemicals, or structurally compromised frameworks exposes responders to life-threatening risks.

To address these deficiencies, autonomous mobile robotics has emerged as a crucial paradigm shift, removing human personnel from immediate danger zones while deploying targeted, real-time fire suppression vectors directly at the flame front. Project **Phoenix**, engineered at King Salman International University (KSIU), addresses this critical operational gap by developing an Intelligent Mobile Robot designed for autonomous flame location detection, life-safety asset tracking, and automated target suppression. By pairing multi-layered sensor arrays with a mobile robotic platform, the system establishes an automated response mechanism capable of localizing fire boundaries and suppressing active flames without placing human operators in hazardous conditions.

1.2 Problem Statement and Limitations of Prior Approaches

The initial architectural framework of the Phoenix prototype relied on a static, pre-saved environmental occupancy grid map to handle navigational routing. In that approach, the workspace layout had to be fully mapped, compiled, and pre-loaded into the ROS 2 Navigation Stack (Nav2) before beginning any mission execution. Furthermore, because the physical hardware platform lacked wheel encoders to calculate odometry data, the system required an artificial "fake" odometry transform layer (odom -> base_link) to bypass Nav2's initialization checks.

This static navigation configuration introduces several major limitations in real-world application:

- Emergency fire scenarios are highly fluid and unpredictable; structural walls can collapse, debris can block corridors, and unmapped obstacles are frequently introduced. A pre-saved map is incapable of reflecting these real-time environmental mutations, causing global path planners to generate obsolete trajectories that lead to collisions.
- A 4-wheel drive skid-steer mobile chassis maneuvers by generating differential velocities across its wheel banks, relying on wheel slippage to overcome lateral ground friction during turns. Without live sensor feedback to measure this slippage, an open-loop or fake odometry transform causes significant, unbounded spatial tracking drift. The robot quickly loses its localization reference relative to the map coordinate origin, causing it to miss target locations completely.

Consequently, there is an urgent engineering requirement to migrate the platform toward an encoder-free, sensor-driven localization strategy. The system must be capable of tracking spatial displacements through real-time laser scan matches, updating its occupancy grids on the fly, and dynamically recalculating global paths through unknown territory.

1.3 System Architecture Overview & Split-Node Model

To optimize power efficiency and maintain rapid responsiveness under battery-powered mobile constraints, Project Phoenix utilizes a distributed, split-node edge computing topography. High-bandwidth camera polling and deep learning inference over neural network structures require continuous computing cycles. Running these tasks natively on a mobile single-board computer would saturate its central processing unit, triggering severe thermal throttling, dropping control loop frequencies, and draining battery reservoirs prematurely.

The split-node model isolates intensive neural computing from physical control tasks by separating the architecture into two dedicated units operating over a localized network layer:

1. **Stationary Vision Station (Laptop Node):** Functions as the high-level global perception engine. It intercepts an RTSP video stream from a fixed wide-area surveillance camera (TP-Link Tapo C210) to monitor the entire workspace. This station executes a custom PyTorch Global Fire Detection Network (GFD-Net) to identify fire boundaries and a dual Keras classification pipeline to flag human

presence. By pairing these visual findings with ArUco fiducial marker tracking, the vision node converts 2D pixel centroids into 3D physical coordinate coordinates X, Y, streaming the spatial data as a JSON payload over a local MQTT broker.

2. **Mobile Edge Computing Node (Raspberry Pi 4 Model B):** Functions as the onboard execution core. It subscribes to the MQTT coordinate topics, ingests real-time distance scans from an Okdo LiDAR HAT, processes planar laser odometry matching, dynamically maps unknown corridors, and generates pulse-width modulation (PWM) commands to control the locomotion motor drivers and fluid pump relays.

This distributed layout keeps heavy visual compute cycles safely decoupled from the mobile chassis, reducing thermal load and power consumption while ensuring the edge node maintains stable control frequencies for safe navigation.

1.4 Scope of Thesis Work and Technical Contributions

While Project Phoenix represents a collaborative engineering effort, this independent thesis details the structural architecture, low-level embedded programming, and sensor integration managed strictly by the author. The explicit scope of this research comprises:

- **Physical Drivetrain and Frame Assembly:** Engineering a rigid structural chassis utilizing 2020 V-Slot aluminum profiles powered by a parallel-wired 4-wheel drive skid-steer configuration using high-torque JGB37-545 DC geared motors.
- **Split-Rail Power Isolation Infrastructure:** Designing a power distribution network driven by a 22.2V series Li-Po battery bus. This include integrating an XL4015 buck converter step-down stage to form a regulated 5V peripheral line, successfully isolating noisy inductive servo and pump feedback loops from the sensitive logic compute elements of the Raspberry Pi 4.
- **Low-Level Hardware Interface Drivers:** Developing and debugging native C++ and Python hardware interaction scripts—namely motor_controller.py, pump_controller.py, and nozzle_controller.py—within the custom phoenix_control ROS 2 package to map high-level spatial velocity commands (/cmd_vel) into physical bare-metal pin duty cycles.
- **Encoder-Free Localization & Dynamic SLAM Migration:** Completely replacing the obsolete pre-saved map infrastructure with a live, sensor-driven navigation stack.

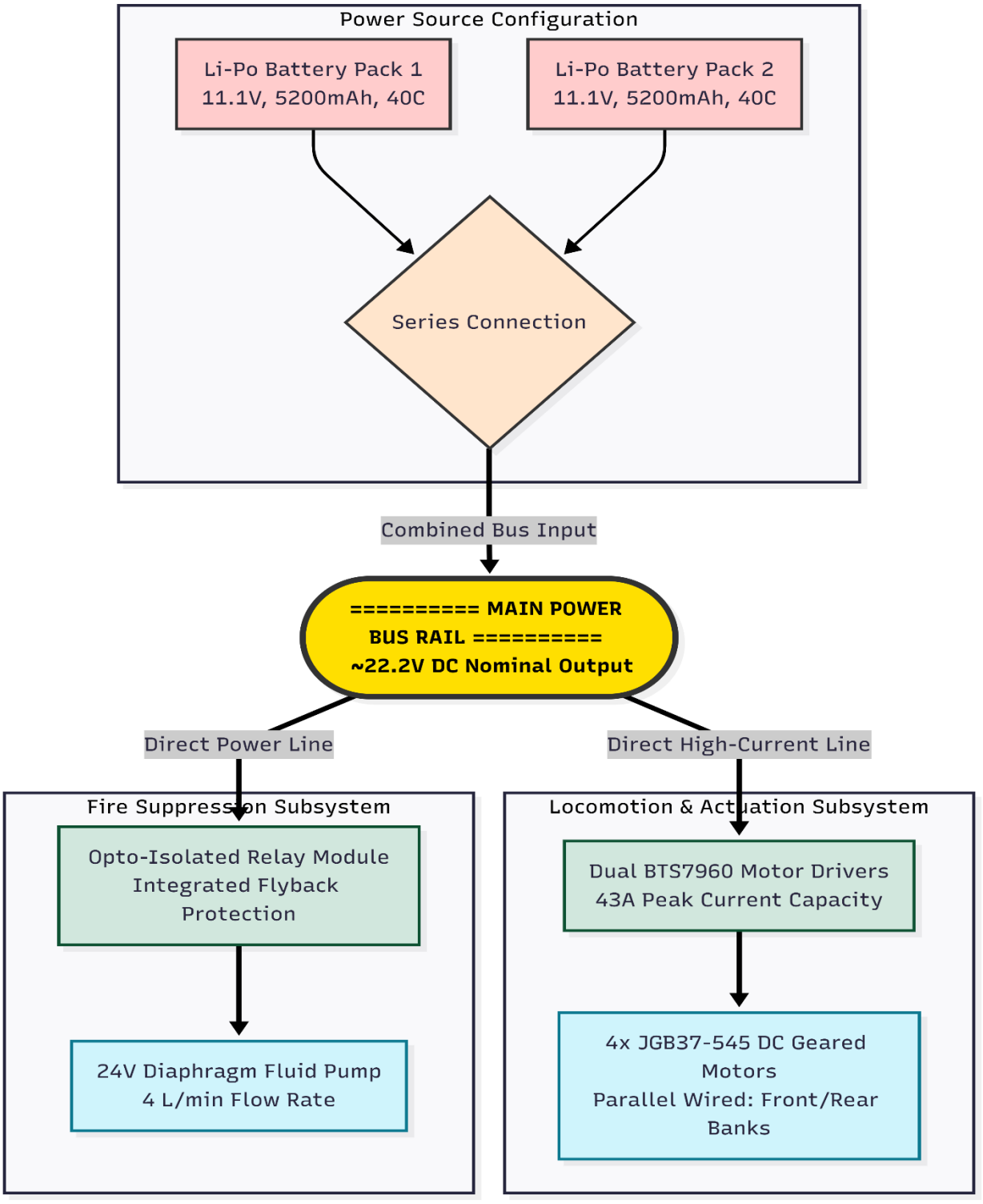
This was achieved by developing a serial packet-decoding pipeline to ingest raw binary UART telemetry from the Okdo DTOF LiDAR HAT, initializing a planar laser odometry estimation node (rf2o_laser_odometry) to compute frame displacements natively, and routing the tracking transformations directly into the slam_toolbox asynchronous mapping loop to generate environmental occupancy grids on the fly.

- **High-Fidelity Simulation Validation:** Creating a precise digital twin model within a custom Unified Robot Description Format (URDF) file and validating path planners within a physics-enabled Gazebo Harmonic environment. This validation process was used to tune asymmetric skid-steer friction factors $(\mu_1, \mu_2)$and verify collision costmap thresholds prior to deployment on physical hardware.

2. Chapter 2: Electrical and Power Systems Design

2.1 Power Source Configuration and Bus Topology

The operational integrity of a mobile robotic firefighting platform depends entirely on the design of its underlying power distribution network. The system requires a high energy density, low weight penalty, and a steady discharge rate to satisfy the current demands of multiple heavy actuation loads working concurrently. To fulfill these metrics, the power subsystem is built around two 11.1V, 5200 mAh Lithium-Polymer (Li-Po) battery packs connected in series. This series configuration generates a high-voltage main power bus supplying a nominal output of approximately 22.2V DC.



2-1 Figure 2.1: Power Distribution and Split-Rail Bus Topology Scheme

The selection of these specific batteries is justified by their high continuous discharge rating of 40C. During rapid acceleration maneuvers or the sudden activation of high-pressure fluid displacement pumps, the robot's powertrain imposes sharp, transient current spikes on the power distribution lines. Standard low-discharge battery chemistry (such as lead-acid or nickel-metal hydride alternatives) suffers from substantial voltage sags under these instantaneous peak conditions. These sags can drop power lines below the operational threshold of connected electronics, causing microcontroller resets and systemic communication timeouts. The 40C continuous discharge headroom safely handles these high transient spikes, guaranteeing stable voltage paths and providing sufficient energy capacity for extended operational runs.

2.2 Split-Rail Power Architecture and Logic Isolation

Inductive mechanical actuators, such as brushed DC motors and solenoid valves, inject significant high-frequency electromagnetic interference (EMI) and transient voltage ripples back into their power rails during high-frequency pulse-width modulation (PWM) switching. If sensitive digital logic cores are tied directly to the same raw electrical lines, this electrical noise can degrade signal transitions, corrupt inter-process bus communications, or trigger fatal processing exceptions. To prevent this interference, this system implements a strict split-rail power isolation architecture:

- **The High-Voltage Actuation Rail:** Operates at the full nominal output of the 22.2V series battery bus. This rail is routed directly to the high-current H-bridge drivers and the fluid suppression pump relay circuits.
- **The Regulated Peripheral Rail:** Built by tapping the main 22.2V bus into an XL4015 adjustable DC-DC step-down buck converter. This high-efficiency step-down converter scales the voltage to a stable 5.0V output line capable of delivering a peak current of 5A. This 5V line is dedicated entirely to running the MG996R nozzle aiming servos, ensuring that sudden current draws from the steering mechanisms cannot pull power away from the primary logic systems.
- **The Isolated Digital Logic Rail:** To safeguard the central processing stack, the onboard Raspberry Pi 4 Model B is powered independently through its native USB-C interface via an external 5.1V, 3A power bank.

By completely removing the single-board computer's power input line from the primary battery bus, the core navigation algorithms and communication services are isolated from transient ripples, voltage drops, and motor stall anomalies.

2.3 Motor Actuation and Driver Integration

The locomotive system of the mobile robot utilizes a four-wheel drive (4WD) skid-steer mechanical drivetrain actuated by four high-torque JGB37-545 DC geared motors. The motors are split symmetrically into left and right propulsion banks. To handle the high continuous current draws required to rotate the chassis against lateral ground friction, the locomotion system is managed by two integrated BTS7960 high-power single-channel H-bridge motor drivers. The front-left and rear-left motor leads are wired in parallel directly into the terminal blocks of Motor Driver 1, while the front-right and rear-right motor leads are wired in parallel into Motor Driver 2.

The choice of the BTS7960 driver over conventional driver modules (such as the L298N) is technically justified by its robust electrical characteristics:

- **Current Handling Capabilities:** The BTS7960 can sustain a continuous output current of up to 43A, providing ample thermal margin when the drivetrain encounters high resistance or high-current stall conditions under full payload weight.
- **Built-in Circuit Protection:** The driver features an intelligent power design that includes integrated over-temperature thermal shutdown, short-circuit protection, and over-current protection loops.
- **High-Frequency Switching Efficiency:** It natively supports high-frequency PWM inputs up to 25 kHz. This high-frequency switching frequency reduces acoustic motor hum and ensures smooth, low-latency control outputs, resulting in highly precise wheel velocity tracking.

2.4 Low-Level Interface Logic and Wiring Allocation

The physical digital interface lines link the Broadcom header of the Raspberry Pi 4 Model B with the driver pins, sensor modules, and relay control inputs. The complete physical wiring mapping, derived from the low-level system configuration files, is explicitly detailed in the reference table below:

System Component	Board Signal Identity	Raspberry Pi Pin	Hardware Target Point	Functional Logic Assignment
Logic Power In	USB-C Power Port	N/A	External Power Bank	Regulated 5.1V / 3A input (Isolated Rail)
Common Ground	GND Line	Physical Pin 6	Breadboard GND Rail	**CRITICAL:** Zero-Volt Reference standard
Logic High Bus	3.3V Logic Reference	Physical Pin 1	Breadboard 3.3V Rail	Ties driver enable pins permanently high
Left Driver H-Bridge	Left Forward PWM	GPIO 17 (Pin 11)	Driver 1 R_PWM Input	Generates forward duty cycle to left motors
Left Driver H-Bridge	Left Reverse PWM	GPIO 27 (Pin 13)	Driver 1 L_PWM Input	Generates reverse duty cycle to left motors
Right Driver H-Bridge	Right Forward PWM	GPIO 25 (Pin 22)	Driver 2 R_PWM Input	Generates forward duty cycle to right motors

System Component	Board Signal Identity	Raspberry Pi Pin	Hardware Target Point	Functional Logic Assignment
Right Driver H-Bridge	Right Reverse PWM	GPIO 23 (Pin 16)	Driver 2 L_PWM Input	Generates reverse duty cycle to right motors
Okdo LiDAR HAT	5V Power Supply	Physical Pin 4	LiDAR Power VCC	Draws 5V power directly from the Pi rail
Okdo LiDAR HAT	Serial Telemetry RX	GPIO 15 (Pin 10)	LiDAR Serial Data TX	Ingests raw binary data streams from sensor
Okdo LiDAR HAT	Core Motor Control	GPIO 14 (Pin 8)	LiDAR Spin Control PWM	Controls rotation speed of scan mirror
Fluid Suppression	Pump Control Signal	GPIO 26 (Pin 37)	Opto-isolated Relay Input	Energizes 24V water pump circuit
Aiming Servo	Nozzle Pan Signal	GPIO 13 (Pin 33)	MG996R 180° Control	DMA-backed hardware PWM (50 Hz control)
Aiming Servo	Nozzle Tilt Signal	GPIO 19 (Pin 35)	MG996R 360° Control	DMA-backed hardware PWM

System Component	Board Signal Identity	Raspberry Pi Pin	Hardware Target Point	Functional Logic Assignment
				(Continuous rotation)

2.5 Logic-Level Compatibility, Isolation, and Grounding Strategies

The Raspberry Pi 4 Model B single-board computer operates strictly at a 3.3V logic threshold across its general-purpose input/output (GPIO) array, meaning any incoming or applied voltage above this level can permanently damage the Broadcom microprocessor core. To interface safely with the high-voltage motor drivers, peripheral servos, and fluid pumps, several signal isolation methodologies are used:

- **Optimized Driver Enable Logic:** The input stages of the BTS7960 high-power motor drivers are fully compatible with 3.3V logic signals, interpreting any input above 2.0V as a valid logical HIGH. To optimize GPIO usage on the Raspberry Pi, the forward and reverse channel enable pins (R_EN and L_EN) for both motor drivers are hard-wired directly to the Pi's 3.3V reference voltage rail. This layout locks both H-bridge channels permanently on, enabling the embedded software driver to manage direction, velocity, and deceleration ramps using only four dedicated PWM control lines.
- **Internal Opto-Isolation Protection:** High-voltage isolation is achieved via high-speed optocouplers built directly into the logic input stages of the BTS7960 modules. This optical isolation breaks the direct physical connection between the driver switches and the Pi's GPIO pins, shielding the processor from harmful back-EMF spikes generated during high-frequency motor reversals. Similarly, the 24V fluid pump is isolated using a dedicated opto-isolated relay module. The 3.3V control signal from GPIO 26 drives an internal LED to trigger a photosensitive receiver on the isolated actuation side. This configuration safely switches the 22.2V high-current bus to the pump motor while clamping inductive flyback spikes inside the relay's protective diode loop.

- **Unified Grounding Strategy:** Even though the electrical system uses isolated, split power rails, a common ground reference is strictly required across all sub-circuits. An explicit ground line securely ties the Raspberry Pi's physical ground pin to the main chassis power ground rail. Without this shared zero-volt baseline, the logic lines between the single-board computer and the motor drivers would begin to float, corrupting the duty-cycle ratios of high-frequency PWM commands and causing unpredictable locomotion behaviors.

3. Chapter 3: Distributed Spatial Transformation and Deep Learning Vision Pipeline

3.1 Asynchronous Multithreaded Frame Capture Architecture

A critical bottleneck in real-time robotic computer vision systems is the frame buffer saturation native to standard synchronous image acquisition pipelines. When leveraging a standard single-threaded loop utilizing OpenCV's VideoCapture class, frame decoding and retrieval are bound to the sequential processing rate of the primary application thread. When computationally heavy deep learning neural networks are introduced into this same thread, the execution latency of model inference creates an immediate telemetry lag. Because network cameras (such as the IP-based surveillance streams) continue to broadcast video data at their hardware frame rate, the underlying operating system buffer quickly accumulates un-retrieved frames. This queue causes a cumulative delay where the robot operates on visual telemetry representing the physical environment several seconds in the past, destabilizing high-speed navigational servo loops.

To completely mitigate this deficiency, the Vision Node software implements an asynchronous multithreaded frame drainage engine leveraging Python's native threading constructs. Upon instantiation, the capture module spawns a dedicated background worker thread completely segregated from the central artificial intelligence pipeline. This thread runs an infinite, non-blocking hardware polling loop whose sole operational assignment is to drain the incoming network video stream buffer at its absolute maximum capacity.

By continuously executing read operations, the background thread deliberately overwrites older image frames held within its local memory space, retaining exclusively the single most recent, uncorrupted frame captured by the lens. When the core convolutional neural network requires an image matrix for inference, it bypasses direct hardware buffer extractions entirely and queries a shallow memory reference directly from the thread's shared variable space. This design limits data access latency to less than 10 ms, effectively decoupling the frequency of visual network ingest from the variable execution speeds of deep learning architectures.

3.2 Tiered Convolutional Neural Network Topology

The high-level Vision Node coordinates environmental safety behaviors and target tracking by simultaneously managing two separate deep learning model hierarchies deployed across independent camera network streams:

1. Global Surveillance Tier (GFD-Net via PyTorch)

The software establishes a persistent connection to the primary wide-area environmental camera (TP-Link Tapo C210) via the Real-Time Streaming Protocol (RTSP) over a local Wi-Fi link. The raw incoming 2K resolution frames (2304×1296 pixels) are programmatically downsampled and transformed into a standardized, normalized 3-channel RGB tensor matching the tensor shape required by the custom Global Fire Detection Network (GFD-Net) implemented in PyTorch.

The GFD-Net structural topography processes the pixel matrix through a sequential network backbone comprising 2D convolutional layers, rectified linear unit (ReLU) activation functions, and max-pooling stages. This structural design downsamples spatial dimensionality while distilling high-order deep feature maps. An adaptive average pooling layer standardizes the grid spatial map to a consistent structural tensor array. This array is parsed by a specialized convolutional detection head that calculates candidate bounding box coordinates, object class labels, and localized activation confidence matrices.

The inference script screens these outputs to isolate the highest confidence detection region. If the extracted confidence metric crosses a hardcoded detection threshold of 50%, the normalized grid anchors are mathematically scaled back to match the original pixel proportions of the raw surveillance stream, overlaying a red flame-boundary bounding box around the fire centroid.

2. Local Perception Tier (Keras Models via TensorFlow)

Concurrently, the Vision Node functions as a parallel processing server for the robot's localized embedded perception layer. It ingests raw video arrays streamed over the network from the mobile edge node’s onboard Raspberry Pi Camera Module V3. The stream matrices are programmatically duplicated into twin identical frames and resized to satisfy the input constraints of two separate binary classification networks constructed using Keras APIs running over a TensorFlow backend:

- **Localized Fire Tracking Pipeline:** The first Keras structure monitors changes in immediate flame proximity within the robot's immediate horizontal field of view to serve as a directional tracking verification layer for the nozzle assembly.
- **Human Presence Overriding Pipeline:** The second Keras network executes a highly optimized binary human-detection classifier. If a human signature is successfully flagged inside this local proximity zone, the script triggers an immediate executive priority override. This protective logic overrides standard navigation targets, forcing the platform to prioritize structural life-safety operations and hazard containment around the human target over baseline fire suppression goals.

3.3 Fiducial Marker Tracking and Coordinate Extraction Logic

To bridge the gap between 2D digital image coordinates and the 3D physical workspace, the vision node processes fiducial marker tracking in tandem with the neural network arrays. The software leverages OpenCV's ArUco marker library, targeting a predefined 4x4 square tracking marker array mounted structurally to the upper surface of the mobile chassis.

The localization algorithm completes mapping through the following structured mathematical workflow:

1. **Geometric Space Definition:** The script initializes a rigid array defining the precise physical 3D coordinates of the tracking marker's corners in its own local coordinate system, established from empirical measurements of the physical marker profile indicating an exact boundary edge of 9.3 cm.
2. **Perspective-n-Point (PnP) Resolution:** Upon visual detection of the marker's corners within a frame, the pixel coordinates are cross-referenced with pre-calibrated internal camera matrix parameters A and distortion coefficients D. The software feeds these geometric constraints into an iterative Perspective-n-Point solver utilizing the SOLVEPNP_IPPE_SQUARE optimization flag, which maximizes orientation and pose accuracy for planar structural markers.
3. **Transformation Matrix Assembly:** The PnP solver outputs a 3D translation vector $tvec$ and a 3D rotation vector $rvec$ describing the exact spatial offset of the robot relative to the camera lens optical center. The script converts the 3-element rotation vector into a 3×3 rotation matrix R via Rodrigues' rotation formula. These variables

are compiled to construct a unified 4×4 homogeneous transformation matrix ($T_{\text{camera}}^{\text{robot}}$):

4. $T_{\text{camera}}^{\text{robot}} = \begin{bmatrix} R_{11} & R_{12} & R_{13} & tvec_x \\ R_{21} & R_{22} & R_{23} & tvec_y \\ R_{31} & R_{32} & R_{33} & tvec_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$

Coordinate Export Processing: By resolving this inverse transform matrix, the exact center of the mobile robot is tracked in continuous real-world 3D coordinate space. When the GFD-Net global network flags a fire pixel centroid, this transformation matrix maps the target's pixel vector back onto the physical ground plane grid. The resulting real-world Cartesian coordinate position payload x, y is formatted into a standardized JSON string and dispatched over the local Mosquitto MQTT broker network to instruct the ROS 2 Navigation stack.

3.4 Flask Video Telemetry Microserver

To enable real-time teleoperation, visual debugging, and remote monitoring by human supervisors, a background Flask web microserver runs concurrent with the primary computer vision loops. The server instantiates dedicated, multi-threaded HTTP network routes (/video_feed_tapo for global surveillance and /video_feed_pi for local chassis perception). These network endpoints are wrapped with explicit Cross-Origin Resource Sharing (CORS) security rules to enable unauthorized blocking prevention when linking to external web-based command center dashboards.

The telemetry script continuously samples the fully annotated OpenCV frame matrices—complete with red bounding box overlays tracking global fire boundaries and classification text labels displaying Keras network predictions. Every frame matrix is programmatically compressed into a high-density JPEG stream before being served over the network via a multipart HTTP response protocol utilizing an x-mixed-replace boundary mime-type. This structure allows any standard user web browser connection to continuously stream the changing frame arrays as a live video feed, presenting real-world telemetry with minimal data transmission overhead and zero image buffer backlog.

4. Chapter 4: Low Level Embedded Abstraction and Velocity Interpolation

4.1 Architectural Overview of the phoenix_control Package

The interface bridging high-level algorithmic navigation with bare-metal actuation is encapsulated within the native ROS 2 package titled phoenix_control. Operating directly on the onboard Raspberry Pi 4 Model B edge-computing node, this package isolates real-time electronic control tasks from the computational overhead of simultaneous localization and mapping. The foundational driver within this layer is the motor_controller node. Written in Python using the rclpy client library, this node abstracts direct register manipulation and Pulse-Width Modulation (PWM) duty-cycle management into an object-oriented execution model.

The primary execution entry point instantiates an asynchronous subscriber loop hooked into the intraprocess communication network. The node listens directly to the standard geometry topic /cmd_vel, capturing incoming geometry_msgs/msg/Twist messages with a depth queue limit of 10 packets. This incoming payload consists of three linear and three angular velocity vectors.

Because the mobile platform is structurally constrained to a planar workspace, the node's callback routine isolates the longitudinal linear velocity translation along the x-axis $msg.linear.x$ and the angular yaw velocity command around the z-axis $msg.angular.z$. These variables are decoupled inside an interrupt-driven callback and passed directly to an internal non-blocking control loop operating at a strict frequency of 20 Hz via a ROS 2 hardware timer.

4.2 Kinematic Modeling and Normalized Velocity Conversion

To physically execute the target velocities generated by the path planners, the linear and angular control trajectories must be translated into individual wheel set tangential velocities. The platform utilizes an integrated 4-wheel drive skid-steer chassis layout where the left and right wheel banks are wired symmetrically in parallel. Consequently, the system is governed by a standard differential-drive kinematic reduction under the assumption that the front and rear wheels on each respective side rotate at identical velocities $\left(v_{\text{front\_left}} = v_{\text{rear\_left}} = v_l\right)$.

The node implements true inverse kinematics equations to determine the target left (v_l) and right (v_r) velocities in meters per second:

$$v_l = v_{\text{linear}} - \left(\frac{\omega_{\text{angular}} \cdot B}{2.0}\right)$$
$$v_r = v_{\text{linear}} + \left(\frac{\omega_{\text{angular}} \cdot B}{2.0}\right)$$

Where B represents the physical track width of the robot chassis, configured within the core execution code to an empirical spacing of 0.35 meters.

Once these individual channel tangential velocities are computed, they must be converted into normalized duty-cycle percentages compatible with open-loop motor driver execution. The controller establishes a theoretical maximum linear velocity constant (V_{max}) of 2.246 meters per second, representing the absolute top speed of the JGB37-545 geared motors operating at 100% PWM power under nominal voltage conditions. The code normalizes the left and right motor commands by dividing the calculated tangential velocities by this maximum threshold:

$$\text{left\_speed} = \frac{v_l}{V_{\text{max}}}$$
$$\text{right\_speed} = \frac{v_r}{V_{\text{max}}}$$

This reduction transforms absolute linear metrics into a floating-point scalar range bounded strictly between $[-1.0 \text{ and } 1.0]$, directly representing the target percentage power of the active drive bus.

4.3 Real-Time Velocity Interpolation and Ramping Algorithm

Directly presenting raw, unmitigated step changes in velocity commands to high-torque DC geared motors introduces severe mechanical and electrical stress into the mobile platform. Instantaneous motor reversals or rapid acceleration requests draw high transient current spikes from the main power rail, inducing extreme thermal loads, causing voltage sags that reset onboard logic elements, and damaging the gear assemblies. Additionally, sudden changes in wheel velocity cause wheel slippage on the ground, introducing large localized tracking errors that compromise the consistency of the laser mapping pipeline.

To resolve these stability issues, the MotorController implementation embeds a discrete real-time velocity interpolation layer directly inside its 20 Hz processing loop. The internal variables track the currently executing velocity states (current_linear and current_angular)

and step them sequentially toward the changing inputs (target_linear and target_angular) using fixed step boundaries per control cycle:

$$\Delta v_{\text{linear}} = 0.05, \quad \Delta \omega_{\text{angular}} = 0.05$$

The node executes this interpolation profile using conditional branch checking inside a dedicated sub-routine named approach_target. The mathematical mechanics of this ramp behavior are modeled as follows:

$$\text{Next Velocity} = \begin{cases} \min(\text{current} + \text{step}, \text{target}), & \text{if current} < \text{target} \\ \max(\text{current} - \text{step}, \text{target}), & \text{if current} > \text{target} \\ \text{target}, & \text{if current} = \text{target} \end{cases}$$

Because the control timer rate is clocked at 0.05 seconds, a step configuration of 0.05 restricts velocity changes to a maximum acceleration rate of 1.0 unit/second^2. This bounds the output acceleration envelope, ensuring it takes exactly 1.0 second to ramp from a dead stop to full maximum power. This smooth ramping behavior protects the mechanical drivetrain elements from sharp transient shocks and eliminates wheel slippage during trajectory tracking.

4.4 Empirical Calibration Subroutines and Protective Scaling

Due to practical physical variances in chassis assembly, manufacturing tolerances in internal gear assemblies, or accidental polarity reversals during lower-level wiring routing, the digital command vectors can cause incorrect locomotion directions. To correct these issues without physical rewiring, the node incorporates hardcoded empirical calibration booleans during initialization:

Python

```
self.swap_left_and_right = True   # Corrects structural asymmetry
self.invert_left         = True   # Resolves left polarity inversion
self.invert_right        = False  # Maintains right nominal state
```

The node processes these parameters sequentially within the control loop execution thread. If swap_left_and_right evaluates to True, the code performs an inline element swap, mapping the computed left scalar to the right channel and vice-versa to correct directional turning errors. If invert_left or invert_right are asserted, the corresponding channel speed is multiplied by a negative scale factor -1 to realign the software command vectors with the actual physical rotation of the motor shafts.

Following these orientation adjustments, a normalization script screens the final output values to prevent pulse-width saturation. If a high angular rotation command combined with a concurrent linear command pushes the computed velocity beyond the physical hardware limit (max_speed > 1.0), a proportional scaling reduction is applied:

$$\text{left\_speed} = \frac{\text{left\_speed}}{\text{max\_speed}}, \quad \text{right\_speed} = \frac{\text{right\_speed}}{\text{max\_speed}}$$

This scaling down preserves the exact intended curvature and angular turning radius of the path trajectory while keeping the peak current draw within the safe operating limits of the motor driver electronics.

4.5 Bare-Metal Pin Abstraction and Fail-Safe Lifecycle Implementation

The final execution stage maps the normalized, calibrated floating-point scalars into physical voltage variations routed through the single-board computer's Broadcom GPIO header array. The node instantiates four distinct PWMOutputDevice interfaces from the optimized gpiozero framework, mapping the left forward channel to GPIO 17, left reverse to GPIO 27, right forward to GPIO 25, and right reverse to GPIO 23.

The core driver commands these hardware devices by executing conditional branch logic blocks within a sub-routine named set_motor. The speed variable is bounded between $[-1.0,1.0]$, and the logic routes signal states based on directional sign rules:

- **Forward Profile Processing ($speed > 0$):** The script updates the forward pin device value directly to match the floating-point velocity scalar, while forcing the paired reverse pin to a strict logical low state $0.0\ V$.
- **Reverse Profile Processing ($speed < 0$):** The forward control pin drops to a zero duty cycle, while the reverse line asserts a positive voltage ratio equal to the absolute value of the negative speed input $-speed$.
- **Neutral Brake Processing ($speed = 0$):** Both the forward and reverse channel pins dump their duty cycle outputs to zero. This immediate voltage drops forces the continuous H-bridge driver channels to ground, electrically locking the locomotion subsystems to prevent floating position drift.

Python

```
def set_motor(self, fwd_pin, rev_pin, speed):
```

```
    speed = max(min(speed, 1.0), -1.0)
    if speed > 0:
        fwd_pin.value = speed
        rev_pin.value = 0.0
    elif speed < 0:
        fwd_pin.value = 0.0
        rev_pin.value = -speed
    else:
        fwd_pin.value = 0.0
        rev_pin.value = 0.0
```

To safeguard the physical system against unexpected errors or container exceptions, the central execution pipeline wraps the application thread lifecycle inside a formal try/except/finally structural safety block. If a keyboard interrupt (KeyboardInterrupt) or system exception is captured, the standard runtime process is stopped. The exit routine forces all four actuation pins (left_fwd, left_rev, right_fwd, and right_rev) to zero output before unmounting and destroying the node container. This fail-safe cleanup mechanism cuts power to the 22.2V motor circuitry, preventing the robot from entering runaway conditions if the higher-level ROS 2 execution stack unexpectedly fails.

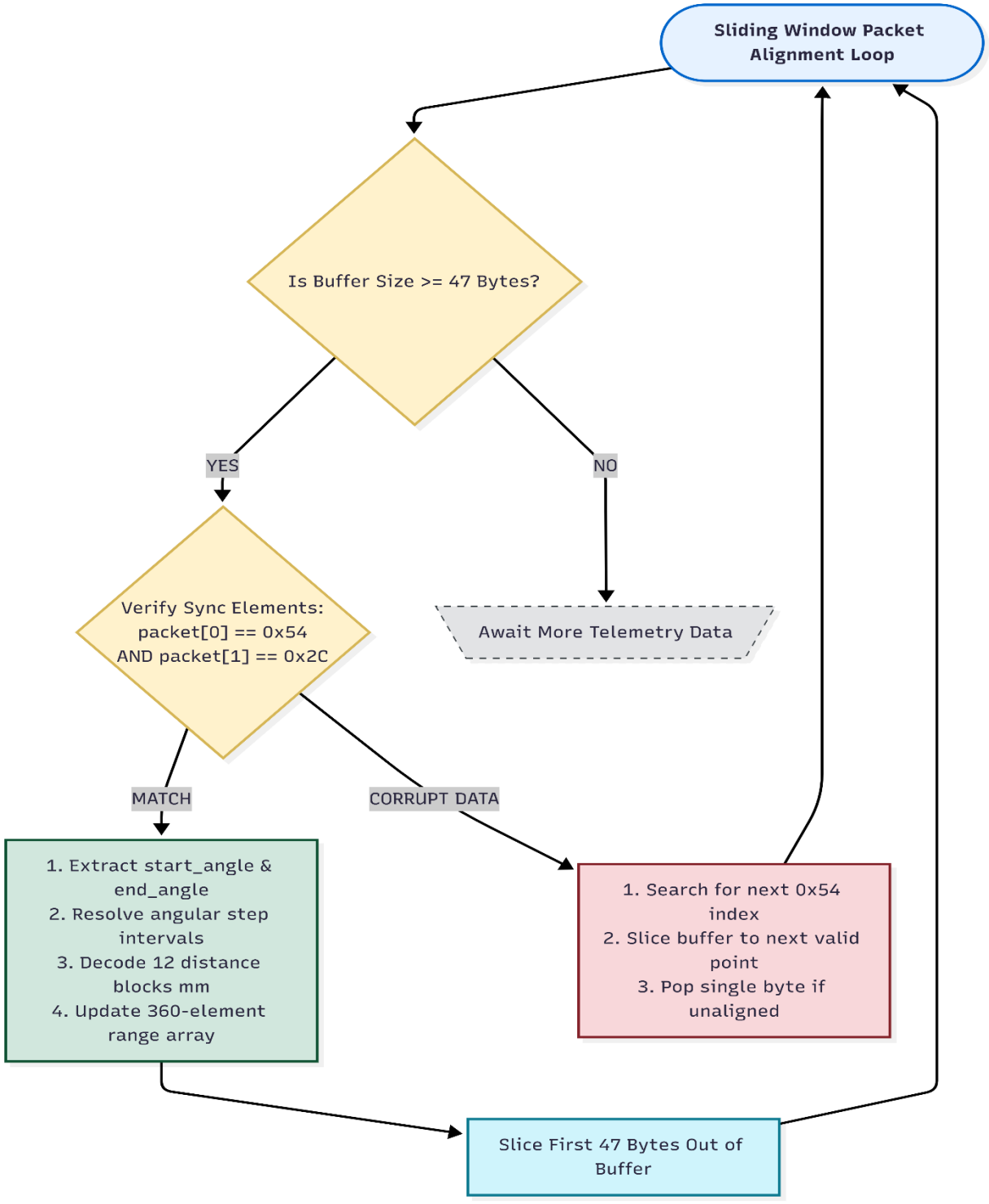
5. Chapter 5: ROS 2 Navigation Stack and LiDAR-Based Dynamic SLAM

5.1 Non-Blocking Serial Data Ingestion and Telemetry Decoding Pipeline

The core performance of the robot's dynamic mapping and navigation algorithms is directly dependent on the latency and stability of its environmental sensor pipeline. The platform integrates an Okdo (LD06) Direct Time-of-Flight (DTOF) LiDAR sensor that continuously outputs distance telemetry across a planar 360° field of view. The sensor transmits raw binary packet payloads to the single-board computer's hardware UART serial port (/dev/ttyAMA0) at a high baud rate of 230,400 bps.

To prevent serial buffer saturation and minimize processing delays, the lidar_publisher node uses a decoupled, multithreaded data ingestion architecture. The node establishes an asynchronous worker thread (read_loop) configured as a persistent daemon thread. This execution model isolates raw serial port reads from the primary ROS 2 runtime thread, allowing data collection to remain uninhibited by downstream messaging overhead. The worker thread polls the port using a non-blocking in_waiting register check. When bytes accumulate, the thread extracts the raw array and triggers the data parsing module. If the serial buffer is empty, the thread enters a brief 5 ms sleep state. This periodic delay limits CPU core utilization, preventing thermal throttling on the Raspberry Pi 4 edge node while preserving processing capacity for mapping.

The incoming byte data is parsed using a sliding window byte-alignment routine implemented inside the parse_lidar_data routine. The serial string streams into a dynamic bytearray buffer. The parsing algorithm processes this array through the following programmatic workflow:



5-1 Figure 5.1: Sliding Window Byte-Alignment and Telemetry Decoding Loop

1. **Header Synchronization and Verification:** The loop evaluates the leading indices of the active array. A valid hardware telemetry packet must begin with a sync header byte of 0x54 and a packet length byte of 0x2C (representing a constant 47-byte block size). If alignment matches, the 47-byte slice is isolated for telemetry decoding.
2. **Unaligned String Correction:** If the leading indices fail to match the header parameters, the script identifies a data corruption or drift state. The node executes an internal recovery block utilizing an .index(0x54) boundary lookup to slice the buffer up to the next potential valid sync marker. If the specific sync header is absent, the buffer is cleared entirely to discard corrupted telemetry.
3. **Telemetry Intercept and Scale Parsing:** In a verified packet, the starting angle (packet[4:6]) and ending angle (packet[42:44]) are extracted as 2-byte little-endian unsigned integers. The resulting values are divided by a hardware scaling factor of 100.0 to convert the data into absolute degrees:

$$\theta_{\text{start}} = \frac{\text{int.from\_bytes}(\text{packet}[4:6],\text{'little'})}{100.0}$$
$$\theta_{\text{end}} = \frac{\text{int.from\_bytes}(\text{packet}[42:44],\text{'little'})}{100.0}$$

4. **Angular Interpolation and Conversion:** The angular differential $\Delta\theta$ between the limits is resolved by accounting for zero-crossing wrap-arounds ($\Delta\theta = \theta_{\text{end}} - \theta_{\text{start}} + 360.0$ if $\Delta\theta < 0$). Each hardware block packs 12 distinct measurement points, meaning the angular sweep is split into 11 separate step intervals $step = \Delta\theta/11.0$.
5. **Distance Extraction and Array Mapping:** The loop steps through the 12 measurement slots, calculating the target angle for each sample point $\left(\theta_i = (\theta_{\text{start}} + i \cdot \text{step}) \pmod{360.0}\right)$. The corresponding distance bytes are extracted from their structural index $idx = 6 + i \cdot 3$, converted from little-endian integers, and scaled from millimeters to meters ($\text{distance}_{\text{m}} = \text{distance}_{\text{mm}}/1000.0$). The derived distance floating-point value is mapped to an internal 360-element array (current_ranges), where the rounded index represents the physical target direction relative to the chassis center line.

The compiled sensor telemetry is broadcast to the ROS 2 ecosystem via a hardware timer scheduled to run at 10 Hz. The callback allocates a standard sensor_msgs/msg/LaserScan data container, assigning the frame_id to lidar_link and using the ROS 2 node clock to generate accurate timestamps. The configuration fields establish a full 2π planar field-of-view using 1° angular resolution steps:

$$\theta_{\text{min}} = 0.0, \quad \theta_{\text{max}} = 2\pi, \quad \Delta\theta_{\text{increment}} = \frac{\pi}{180.0} \text{ rad}$$

The data mask enforces distance minimum and maximum boundaries of 0.35 m and 10.0 m, respectively. Filtering measurements below 0.35m prevents the robot's own chassis profiles and equipment enclosures from being interpreted as obstacles. To prevent race conditions from concurrent background thread modifications during message transmission, the node publishes a duplicate copy of the array. This scan dataset is broadcast onto the /scan topic using an optimized sensor QoS profile (qos_profile_sensor_data), which drops dropped packets to optimize network data transmission.

5.2 Planar Laser Odometry Estimation (rf2o_laser_odometry)

Because this mobile platform is engineered without hardware wheel encoders, traditional locomotion tracking methods are unavailable. To provide the relative coordinate transformations required by the navigation stack, the software uses the rf2o_laser_odometry planar execution framework. This node estimates the displacement transforms of the robot frame (odom -> base_link) by executing dense scan matching across consecutive LiDAR scan profiles.

The coordinate frame transformation tree is configured via the laser_odom.launch.py script. First, the robot_state_publisher node processes the parameters detailed inside the custom phoenix.urdf description file. This publishes the static transform coordinates linking the physical components of the robot (e.g., establishing the exact 3D translation vector from base_link up to the lidar_link scanning plane).

Concurrently, the rf2o_laser_odometry_node subscribes to the /scan topic. It maps spatial point displacements at an update frequency of 10.0 Hz and publishes the resulting estimates to the /odom topic. By clearing the init_pose_from_topic string parameter, the odometry node initializes immediately upon execution without waiting for an external ground truth baseline. The module asserts the publish_tf = True parameter to broadcast the live odom -> base_footprint transform link directly to the core coordinate tree. This scan-

matching strategy provides a stable, encoder-free localization reference that avoids the positional drift errors common to uncalibrated open-loop steering configurations.

5.3 Asynchronous Online Mapping Parameters via SLAM Toolbox

The spatial point streams feed directly into the slam_toolbox online mapping server to generate real-time environmental occupancy grids, eliminating the requirement for pre-saved factory or warehouse maps. The mapping node is managed via parameters detailed inside the mapper_params_online_async.yaml configuration file:

YAML

```yaml
slam_toolbox:
  ros__parameters:
    solver_plugin: solver_plugins::CeresSolver
    ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
    ceres_preconditioner: SCHUR_JACOBI
    ceres_trust_strategy: LEVENBERG_MARQUARDT

    # ROS Frames
    odom_frame: odom
    map_frame: map
    base_frame: base_footprint
    scan_topic: /scan
    mode: mapping # Dynamic mapping

    # Performance optimizations for Raspberry Pi 4
    map_update_interval: 2.0
    resolution: 0.05
    max_laser_range: 10.0
    minimum_time_interval: 0.1
    transform_timeout: 0.2
    tf_buffer_duration: 30.0
```

The optimization pipeline utilizes the Ceres non-linear optimization solver (solver_plugins::CeresSolver), implementing a SPARSE_NORMAL_CHOLESKY linear solver paired with a SCHUR_JACOBI preconditioner and a LEVENBERG_MARQUARDT trust strategy. This solver architecture ensures fast convergence when updating structural loop closures and tracking node landmarks.

To balance mapping quality with the physical compute limitations of the single-board computer, the map_update_interval parameter is configured to 2.0 seconds. This parameter reduces CPU load by introducing a small, controlled delay between map updates, leaving sufficient computing overhead for path planning and motor control. The resolution parameter defines a precise grid resolution of 5 cm $0.05\ m$, which allows the global and local costmap layers to reliably identify tight passageways and avoid small structural debris.

5.4 Event-Driven Action Client Integration and Preemption Control

The mqtt_nav_client node provides the core coordination link between high-level visual targets and low-level physical path execution. The class handles network communications by embedding an active Paho MQTT subscriber client within its container, utilizing the updated CallbackAPIVersion.VERSION2 interface standards. Upon initialization, the client connects to the local broker at localhost:1883 and subscribes to the ambers/robot/navigation/target and ambers/robot/pump messaging topics.

When target coordinate payloads arrive over the network, the subscriber callback parses the payload string. If a manual override or emergency message is received on the ambers/robot/pump topic with an activation flag set to True, the node executes a protective shutdown block. It checks for any active navigation goal handles, and if found, invokes an asynchronous cancel request (cancel_goal_async()) to immediately halt vehicle motion before publishing a True flag on the local target_reached topic to activate the pump circuitry.

Python

```
if msg.topic == "ambers/robot/pump":
    trigger = data.get("activate", False)
    if trigger:
```

```
        self.get_logger().info("Received MQTT Pump trigger. Activating
pump...")
        if self.current_goal_handle is not None:
            self.get_logger().info("Canceling active Nav2 goal before
starting pump...")
            self.current_goal_handle.cancel_goal_async()

        msg_out = Bool()
        msg_out.data = True
        self.pump_trigger.publish(msg_out)
    return
```

For standard target tracking operations, the client extracts the Cartesian fields x and y from the incoming JSON payload. Because the high-level vision node streams target updates continuously, directly forwarding every coordinate change to the navigation stack would oversaturate the global path planner, creating continuous goal preemption loops that lead to jerky, unstable movements. To resolve this trajectory instability, the node incorporates an optimization filter that measures the Euclidean distance change between the new coordinates and the currently executing goal location:

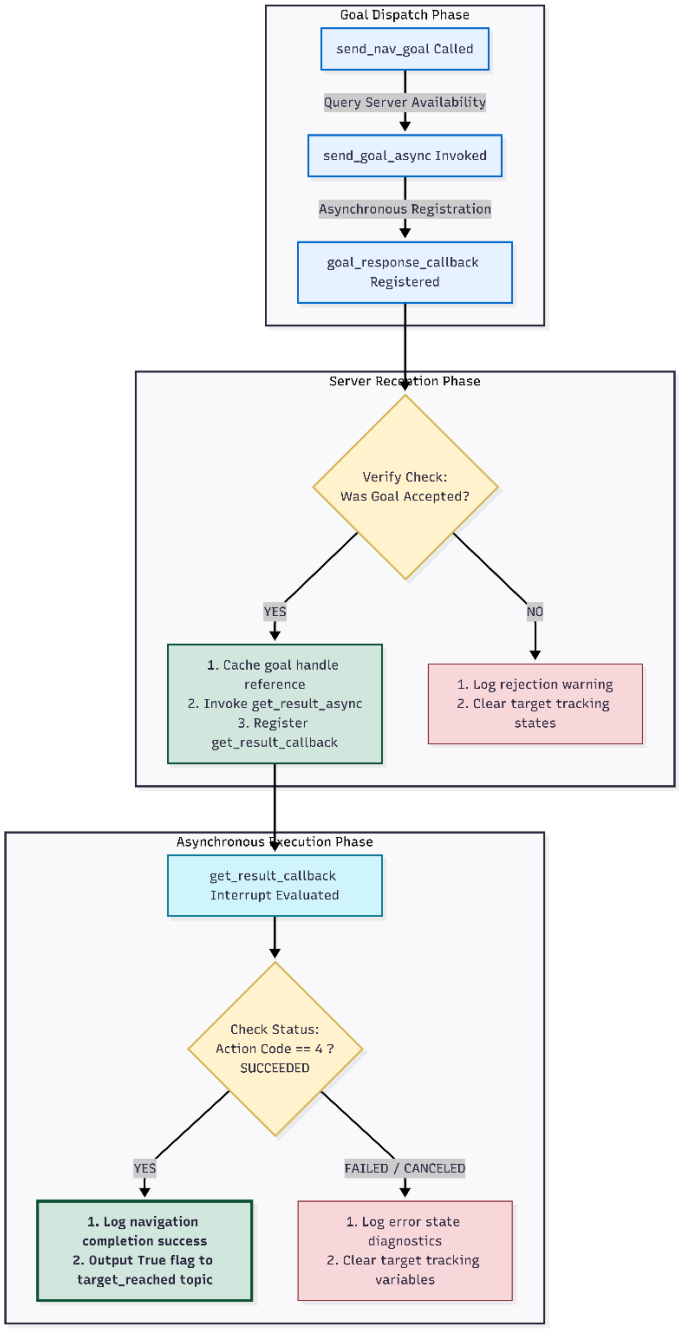
$$d = \sqrt{\left(x_{\text{target}} - x_{\text{active}}\right)^2 + \left(y_{\text{target}} - y_{\text{active}}\right)^2}$$

If the calculated spatial shift falls below a hardcoded dead-band threshold of 0.10 meters $(10\ cm)$, the node discards the incoming MQTT payload. This optimization prevents redundant preemption requests and allows the robot to maintain a stable, continuous trajectory.

If the target coordinates exceed the dead-band threshold, the node initiates an action transaction using an asynchronous ROS 2 Action Client linked to the Maps_to_pose action server. The node populates a MapsToPose.Goal message container, mapping the coordinate parameters directly into the global reference framework (frame_id = 'map'). Crucially, the goal header timestamp is explicitly assigned to exactly zero (Time().to_msg()). This design decision prevents coordinate tree transform errors caused by network

propagation latency or clock drift between the processing nodes, ensuring the navigation stack can immediately process the target vector.

The action client execution lifecycle is managed via nested asynchronous callback layers:



5-2 Figure 5.2: Event-Driven Action Client Transaction and Lifecycle State Machine

The node evaluates the return status within the final execution callback (get_result_callback). If the path planner successfully navigates the chassis into the target goal tolerance area, the action server returns an integer status code of exactly 4 (SUCCEEDED). When this status condition is met, the node outputs a boolean True message onto the localized target_reached topic. This message alerts the onboard relay node to trigger the 24V fluid pump, automating the transition from autonomous navigation to target fire suppression.

6. Chapter 6: ROS 2 Software Integration and Control Nodes

6.1 Real-World Environmental Deployment and Testing Baseline

To validate the operational capabilities of the decentralized software stack and the encoder-free localization framework, the physical platform was deployed within an unstructured workspace layout. Testing a 4-wheel drive skid-steer platform in a real-world setting requires establishing specific environmental baselines due to the high lateral friction encountered during differential turning maneuvers. The physical platform, with a fully loaded 4-liter water capacity tank, features a gross vehicle weight of approximately 5.4 kg. This payload mass increases the normal force acting on the contact points, highlighting the need for an odometry system that does not rely on physical wheel rotation data.

The physical validation tests were conducted across a planar concrete workspace floor area. Prior to running any software routines, the wide-area surveillance camera (TP-Link Tapo C210) was securely mounted overhead to provide uninhibited visual coverage of the entire testing area. The 4x4 ArUco fiducial chassis array was calibrated to its exact structural boundary measurement of 9.3 cm to ensure accurate real-world scale calculation inside the camera coordinate transformation loop. Wireless network connectivity was verified by establishing a shared local subnet router link. This local subnet allowed continuous network communication between the stationary laptop vision node and the mobile single-board computing core without requiring an active internet connection.

6.2 Low-Level Process Launch Order

To initialize the complete decentralized architecture on the physical robot, ten separate terminal shell processes are opened sequentially on the Raspberry Pi 4 edge core via Secure Shell (SSH) connections. Every active shell environment enforces a shared network identifier (export ROS_DOMAIN_ID=30) and sources the local workspace setup file (source ~/ambers_ws/install/setup.bash) to ensure reliable intra-process message routing.

The step-by-step launch sequence must be executed in the following order:

Terminal 1: Lidar Driver and Publisher

Establishes the non-blocking hardware interface with the serial port, captures raw binary streams from /dev/ttyAMA0 at 230,400 bps, and publishes the decoded arrays onto the /scan topic.

Bash

```
export ROS_DOMAIN_ID=30
source ~/ambers_ws/install/setup.bash
ros2 run phoenix_control lidar_publisher
```

Terminal 2: Laser Odometry and Coordinate Transforms

Launches the rigid URDF description profiles and executes high-speed scan matching to calculate continuous spatial displacements, broadcasting the standard dynamic coordinate transform (odom -> base_link).

Bash

```
export ROS_DOMAIN_ID=30
source ~/ambers_ws/install/setup.bash
ros2 launch phoenix_description laser_odom.launch.py
```

Terminal 3: Asynchronous Online Mapping (SLAM Toolbox)

Initializes the Ceres optimization solver to build an environmental occupancy map on the fly and broadcasts the primary coordinate transform tree link (map -> odom).

Bash

```
export ROS_DOMAIN_ID=30
source ~/ambers_ws/install/setup.bash
ros2 launch slam_toolbox online_async_launch.py
slam_params_file:=/home/ambers/ambers_ws/src/phoenix_description/config/ma
pper_params_online_async.yaml use_sim_time:=False
```

Terminal 4: Nav2 Navigation Stack

Loads the behavioral recovery nodes, local costmap thickeners, and the global path planner pipelines.

Bash

```
export ROS_DOMAIN_ID=30
source ~/ambers_ws/install/setup.bash
ros2 launch nav2_bringup navigation_launch.py use_sim_time:=False
```

Terminal 5: Locomotion Motor Controller

Runs the inverse kinematics driver node to parse target velocity matrices (/cmd_vel) and translate them into hardware PWM duty cycles managed via the gpiozero framework.

Bash

```
export ROS_DOMAIN_ID=30
source ~/ambers_ws/install/setup.bash
ros2 run phoenix_control motor_controller
```

Terminal 6: MQTT Navigation Client Bridge

Starts the event-driven action client link to convert incoming MQTT target JSON payloads into asynchronous Nav2 action goals.

Bash

```
export ROS_DOMAIN_ID=30
source ~/ambers_ws/install/setup.bash
ros2 run phoenix_control mqtt_nav_client
```

Terminal 7: Suppression Pump Controller

Starts the subscriber node dedicated to managing the opto-isolated single-channel mechanical relay on GPIO pin 26.

Bash

```
export ROS_DOMAIN_ID=30
source ~/ambers_ws/install/setup.bash
ros2 run phoenix_control pump_controller
```

Terminal 8: Nozzle Steering Controller

Launches the driver script dedicated to sending hardware PWM signals to steer the pan-tilt servo motors.

Bash

```
export ROS_DOMAIN_ID=30
source ~/ambers_ws/install/setup.bash
ros2 run phoenix_control nozzle_controller
```

Terminal 9: Local Camera Stream Publisher

Runs a background shell script to initialize the hardware capture pipeline on the Raspberry Pi Camera Module V3, streaming real-time local images over the network.

Bash

```
cd ~/ambers_ws
bash ~/ambers_ws/scripts/start_pi_camera_stream.sh
```

Terminal 10: MQTT Manual Override Bridge

Launches a manual control bridge to listen for direct steering velocity updates from the web-based command center interface.

Bash

```
export ROS_DOMAIN_ID=30
source ~/ambers_ws/install/setup.bash
ros2 run phoenix_control mqtt_motor_bridge
```

6.3 Local Mosquitto WebSocket Configuration

The web-based command center dashboard requires a WebSocket interface to stream telemetry and commands directly inside standard browser environments. To support this connection alongside native ROS 2 nodes, the local Mosquitto broker configuration on the Raspberry Pi must be updated.

A dedicated configuration file is generated at /etc/mosquitto/conf.d/websocket.conf with the following networking parameters:

```
# Standard MQTT Port for ROS 2 Nodes

listener 1883

protocol mqtt

allow_anonymous true


# WebSocket Port for Web UI Command Center

listener 9001

protocol websockets

allow_anonymous true
```

After deploying the parameters, the broker service is restarted via systemd control hooks (sudo systemctl restart mosquitto). Port assignment verification is confirmed by executing a network status check (ss -tlnp | grep 9001), ensuring that the WebSocket listener is active and ready to handle incoming browser connections.

6.4 Laptop Execution and Visualization Environments

Once the mobile edge hardware processes are active, the operator configuration steps are completed on the stationary laptop vision station:

1. **Vision Inference Node Initialization:** A terminal window is launched to execute the primary computer vision architecture:

Bash

```
python3 vision.py
```

This script triggers the multi-threaded video stream grabber, opens the RTSP wide-area camera feed, starts the PyTorch GFD-Net fire localization pipeline, and begins tracking the ArUco marker coordinates.

2. **Web Command Center Activation:** The user opens the control file Phoenix_Web_Command_Center/index.html inside a standard web browser, navigates to the **Settings** panel, assigns the **BROKER WS** line to ws://<PI_IP>:9001/mqtt (injecting the active IP address of the Pi), and clicks the **CONNECT** icon. Switching the system to **MANUAL** mode grants immediate joystick override capabilities to control the locomotion wheels, nozzle servos, and pump relays.
3. **Live Map Visualization via RViz2 (Optional):** To monitor path generation and mapping consistency, the laptop can hook into the shared ROS 2 network by matching the domain identifier line (export ROS_DOMAIN_ID=30) and running the primary visualization toolkit:

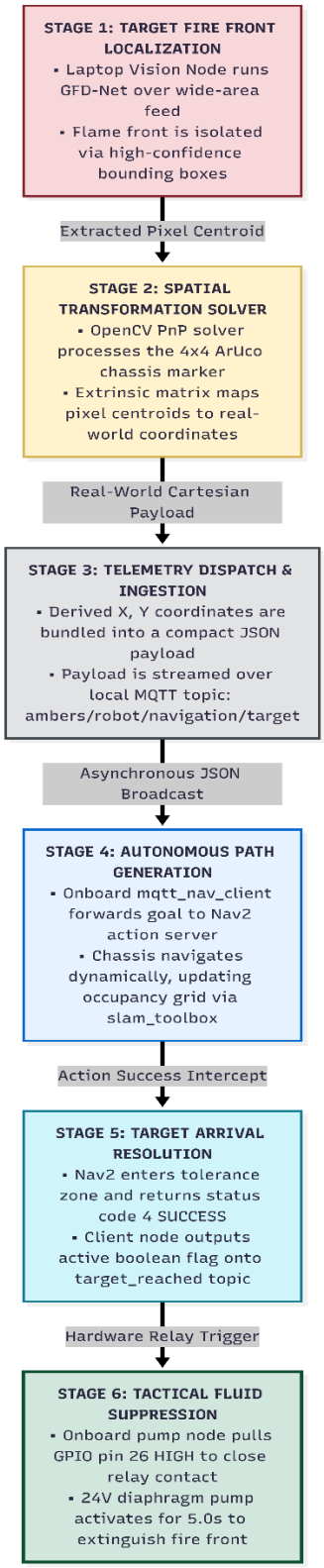
Bash

```
rviz2
```

Within the RViz2 graphical interface, the **Fixed Frame** is configured to map, a **Map Display** is assigned to subscribe to the /map topic, a **RobotModel Display** is added to reflect the chassis link orientations, and a **LaserScan Display** is linked to the /scan topic to view live proximity point clouds.

6.5 Full Physical Mission Workflow Walkthrough

When the platform is deployed in a dynamic environment, it completes its mission sequence through an integrated six-stage execution workflow:


STAGE 1: TARGET FIRE FRONT LOCALIZATION
▪ Laptop Vision Node runs GFD-Net over wide-area feed
▪ Flame front is isolated via high-confidence bounding boxes
Extracted Pixel Centroid
STAGE 2: SPATIAL TRANSFORMATION SOLVER
▪ OpenCV PnP solver processes the 4x4 ArUco chassis marker
▪ Extrinsic matrix maps pixel centroids to real-world coordinates
Real-World Cartesian Payload
STAGE 3: TELEMETRY DISPATCH & INGESTION
▪ Derived X, Y coordinates are bundled into a compact JSON payload
▪ Payload is streamed over local MQTT topic: ambers/robot/navigation/target
Asynchronous JSON Broadcast
STAGE 4: AUTONOMOUS PATH GENERATION
▪ Onboard mqtt_nav_client forwards goal to Nav2 action server
▪ Chassis navigates dynamically, updating occupancy grid via slam_toolbox
Action Success Intercept
STAGE 5: TARGET ARRIVAL RESOLUTION
▪ Nav2 enters tolerance zone and returns status code 4 SUCCESS
▪ Client node outputs active boolean flag onto target_reached topic
Hardware Relay Trigger
STAGE 6: TACTICAL FLUID SUPPRESSION
▪ Onboard pump node pulls GPIO pin 26 HIGH to close relay contact
▪ 24V diaphragm pump activates for 5.0s to extinguish fire front

6-1 Figure 6.1: Comprehensive Six-Stage Physical Mission Execution Workflow

6.6 Conclusion and Future Research Directions

The development and implementation of Project **Phoenix** successfully validates the use of a decentralized split-node architecture for autonomous mobile fire suppression. Offloading resource-intensive computer vision models and multi-layered deep learning inference to a stationary processing station effectively minimizes power consumption and thermal load on the mobile edge node. This optimization preserves critical computing overhead on the single-board computer, enabling stable, low-latency control loops for real-world navigation.

Crucially, migrating the platform to an encoder-free, sensor-driven dynamic mapping stack solves a fundamental operational limitation of previous iterations. By combining dense scan matching via rf2o_laser_odometry with the asynchronous mapping loops of the slam_toolbox, the robot can accurately navigate entirely unknown environments. This approach completely eliminates the unbounded positional tracking drift common to uncalibrated open-loop steering models without requiring physical wheel encoder hardware. The resulting platform successfully automates the deployment sequence from initial remote visual localization to target fluid suppression, establishing a reliable, extensible baseline for autonomous emergency response systems.

Future iterations of this research will focus on two primary enhancements:

- **Hardware-Based Sensor Fusion:** Integrating a low-cost micro-electromechanical (MEMS) Inertial Measurement Unit (IMU) over an I2C communication bus. Fusing high-frequency angular acceleration data with the planar laser scan matching via an Extended Kalman Filter (EKF) will improve tracking accuracy during sudden acceleration adjustments or minor wheel slip events.
- **Active 3D Volume Mapping:** Expanding the 2D laser scan paradigm into 3D space by mounting the LiDAR HAT on a continuous servo pitch mechanism or integrating a depth camera. This enhancement will enable the robot to generate comprehensive 3D point cloud maps, allowing the navigation stack to safely identify overhanging hazards and complex structural debris in multi-level industrial environments.

References

1. Ren, X., Qu, K., Guo, J., Yin, H. and Huang, B., 2025. Research on accurate fire source localization and seconds-level autonomous fire extinguishing technology. *Scientific Reports*, *15*(1), p.17135.
2. Yuvaraj, R., Bhalerao, S.A., Murugesan, K., Vellaiyan, S. and Van Minh, N., 2025. Real-Time Fire Detection and Suppression System Using YOLO11n and Raspberry Pi for Thermal Safety Applications. *Case Studies in Thermal Engineering*, p.107159.
3. Raspberry Pi Foundation, *Raspberry Pi 4 Model B Product Brief*, Raspberry Pi Ltd., 2023.
4. Raspberry Pi Foundation, *Raspberry Pi 4 Model B Datasheet and Hardware Documentation*, Raspberry Pi Ltd., 2023.
5. Sony Semiconductor Solutions Corporation, *IMX708 CMOS Image Sensor Datasheet*, Sony Corp., 2022.
6. Raspberry Pi Foundation, *Camera Module 3 Technical Documentation*, Raspberry Pi Ltd., 2023.
7. TP-Link Technologies Co., Ltd., *Tapo C210 Pan/Tilt Home Security Wi-Fi Camera Datasheet*, TP-Link, 2022.
8. XLSEMI, *XL4015 5A Adjustable Step-Down DC-DC Converter Datasheet*, XLSEMI Microelectronics, 2020.